SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	**Assessment:**	Assessment Tool 2 – Flowchart-To-Code Conversion
Weightage:	35% of CIA	**Total Marks:**	100
CO5 Statement:	*Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.*		
Student Name:	N. Adi Narayana	**Reg. No.:**	192525346
Section / Batch:	AIML	**Date:**	02/09/2026

Code of Conduct

I N. Adi Narayana (192525346) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: Adi

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
	GRAND TOTAL:		**100 Marks**

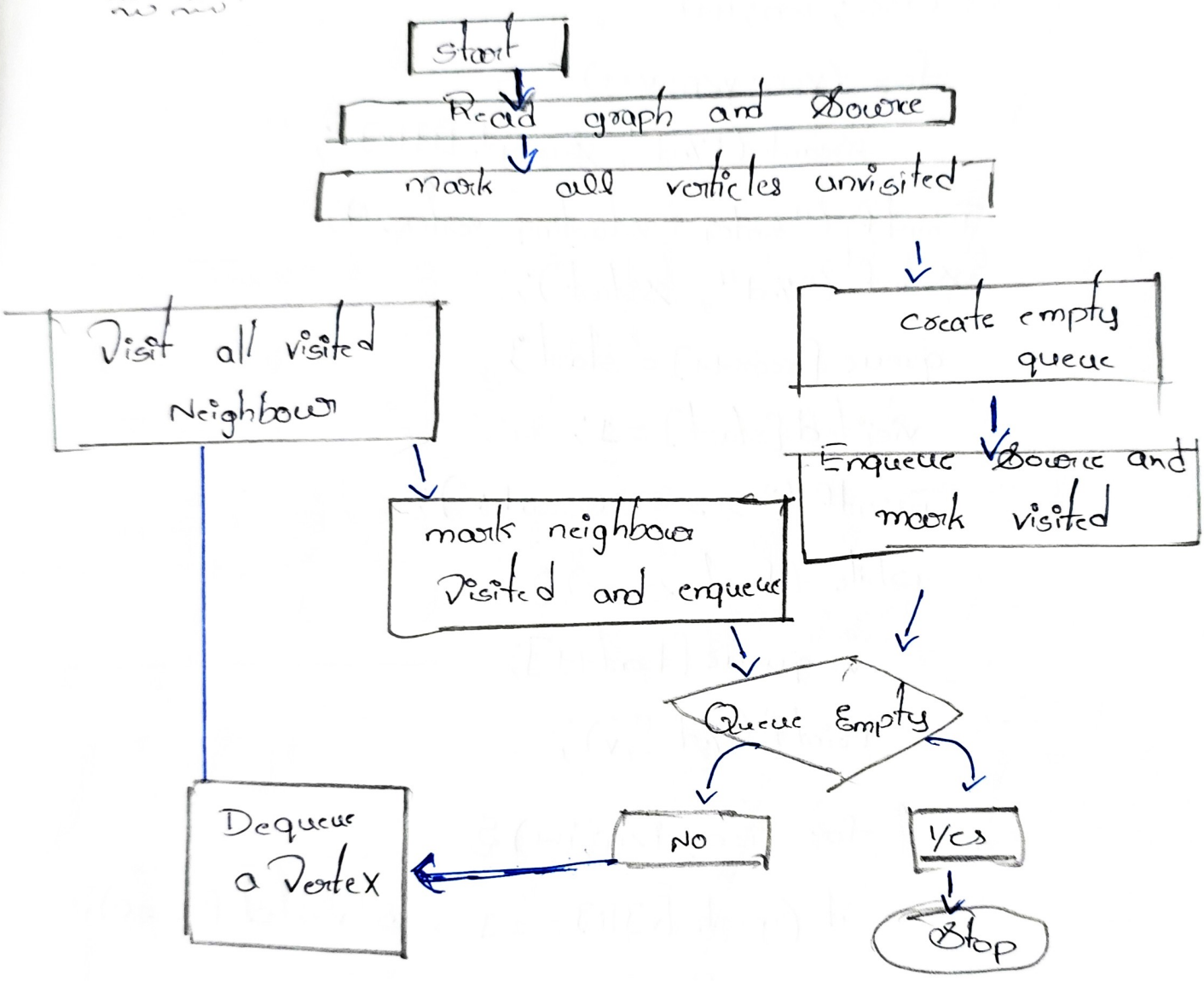
Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Convert the flow chart for Breadth first Search Traversal into code:

flowchart:



C-Program

Code:

```
# include <stdio.h>
    int main () {
    int graph [10][10], visited [10] = {0};
    int queue [10], front = 0, rear = 0;
    int n, start, i, v;
```

```
Printf ("Enter number of vertices:");
scanf ("%d", &n);
Printf ("Enter Adjancey matrix :\n");
for (i=0; i<n; i++)
    for (v=0; v<n; v++)
        scanf ("%d", &graph [i] [v]);
Printf ("Enter starting vertex:");
scanf ("%d", &start);
queue [rear++] = start;
visited [start] = 1;
Printf ("BFS Traversal :");
while (front < rear) {
    v = queue [front++];
    Printf ("%d ", v);
    for (i=0; i<n; i++) {
        if (graph [v] [i] == 1 && visited [i] == 0) {
            queue [rear] = i;
            visited [i] = 1;
        }
    }
}
return 0;
}
```

Input:

5

0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0

starting vertex: 0

Output

BFS Traversal : 0 1 2 3 4

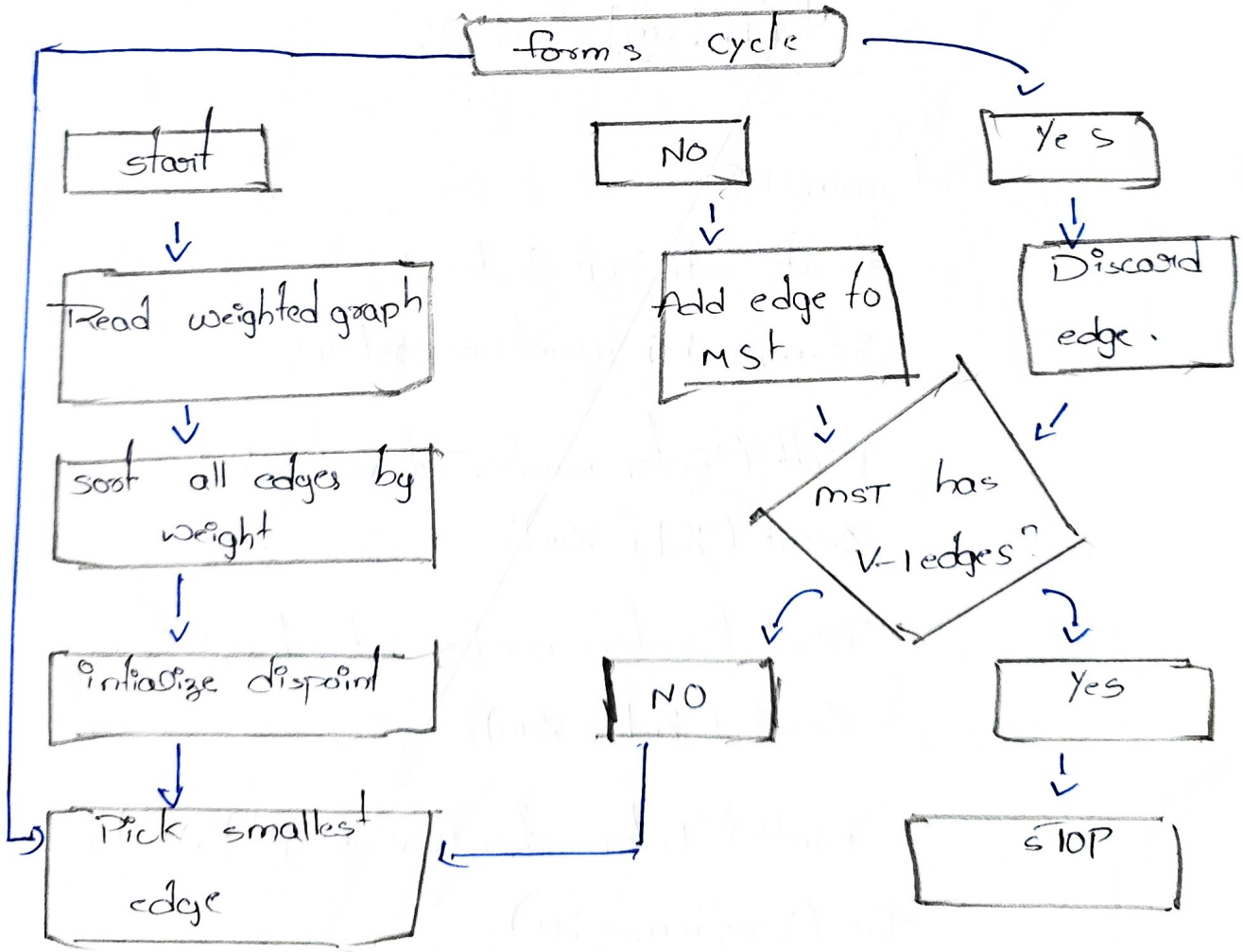
Complexity:

Time : $O(V^2)$

Space : $O(V)$

---

(2) Convert the flowchart for kruskal's algorithm into code.



Code:

```
#include <stdio.h>
struct edge {
    int u, v, w;
};
int parent[20];
int find(int x) {
    while (parent[x] != x)
        x = parent[x];
    return x;
}
void unionset(int a, int b) {
    parent[find(a)] = find(b);
}
int main() {
    struct Edge e[30], temp;
    int n, m, i, j, count=0, cost=0;
    Printf("Enter number of vertices: ");
    Scanf("%d", &n);
    Printf("Enter number of edges: ");
    Scanf("%d", &m);
    Printf("Enter edges (u v weight):\n");
    for (i=0; i<m; i++)
```

```
        Scanf ("%d%d%d", &e[i].u, &e[i].v, &e[i].w);
    for (i=0; i<n; i++)
        Parent[i] = i;
    for (i=0; i<m-1; i++)
        for (j=i+1; j<m; j++)
            if (e[i].w > e[j].w) {
                temp = e[i];
                e[i] = e[j];
                e[j] = temp;
            }
    Printf ("Edges in MST :\n");
    for(i=0; i<m && Count < n-1; i++) {
        if (find (e[i].u) != find (e[i].v)) {
            Printf ("%d-%d = %d\n", e[i].u, e[i].v, e[i].w);
            Cost += e[i].w;
            Union Set (e[i].u, e[i].v);
            Count ++;
        }
    }
    Printf ("Minimum cost = %d\n", Cost);
    return 0;
}
```

Input:

4

5

0 1 10

0 2 6

0 3 5

1 3 15

2 3 4

Output:

Edges in MST : 2-3=4

0-3=5

0-1=10

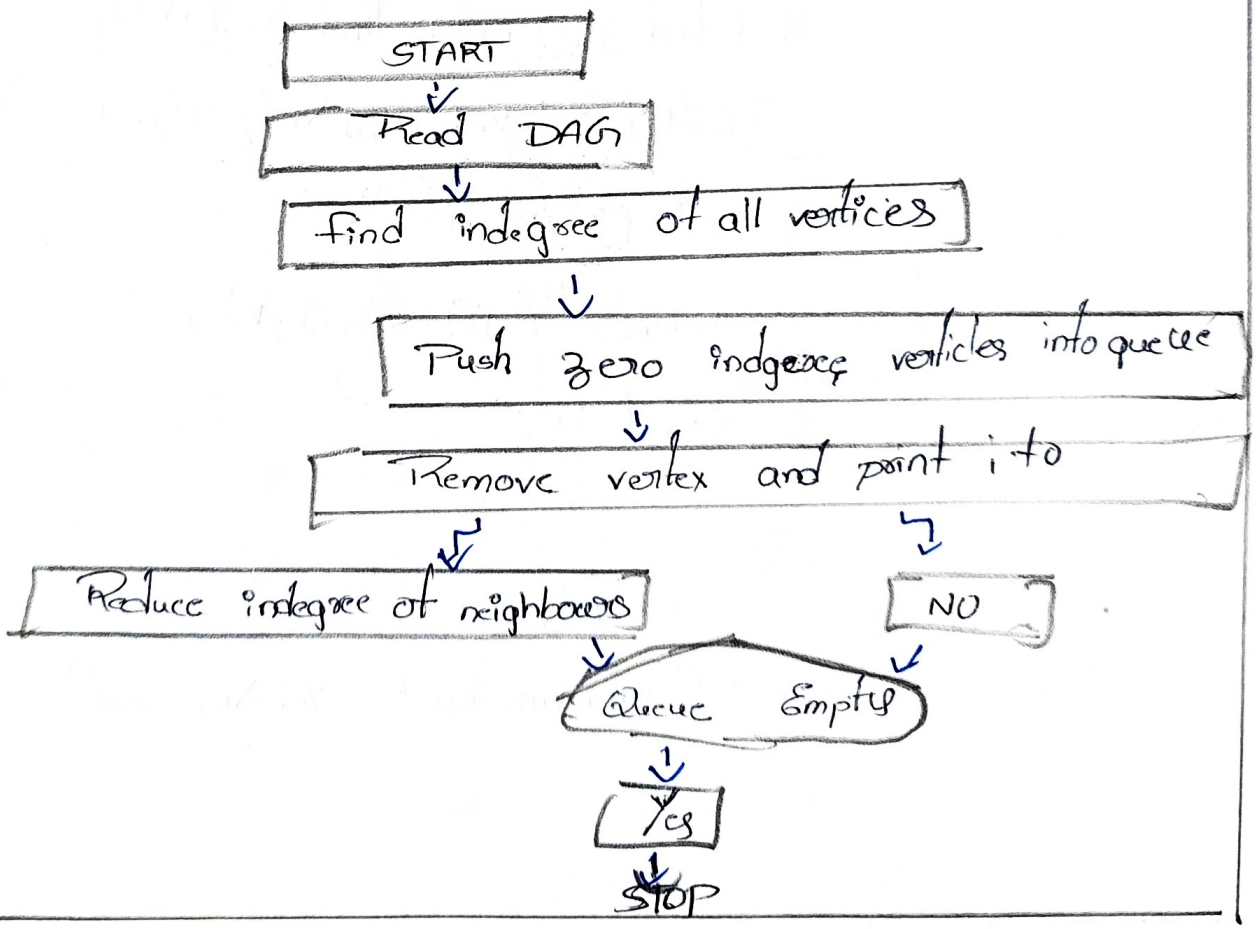
Minimum cost=19

Time : $O(E^2)$ Using simple sorting

Space : $O(V)$

---

(3) Convert the flowchart for topological sort into code



Code:

```
#include <stdio.h>
int main(){
    int graph[10][10], indegree[10]={0};
    int queue[10], front=0, rear=0;
    int n, i, j, v, count=0;

    Printf("Enter number of vertices:");
    Scanf("%d", &n);
    for(i=0;i<n;i++)
        for(j=0;j<n;i++)
            Scanf("%d", &graph[i][j]);
    for(i=0;i<n;i++)
        for(j=0;j<n;j++)
            if(graph[i][j])
                indegree[j]++;
    for(i=0;i<n;i++)
        if(indegree[i]==0)
            queue[rear++]=i;
    Printf("Topological Order:");
    while(front<rear){
        v=queue[front++];
        Printf("%d ",v);
        count++;
        for(j=0;j<n;j++){
            if(graph[v][j]){
                indegree[j]--;
```

```
            if (indegree[j]==0)
                queue[rear++]=j;
        }
    }
}
if (count!=n)
    Printf("\n Cycle exists!");
    return 0;
}
```

Input:

4

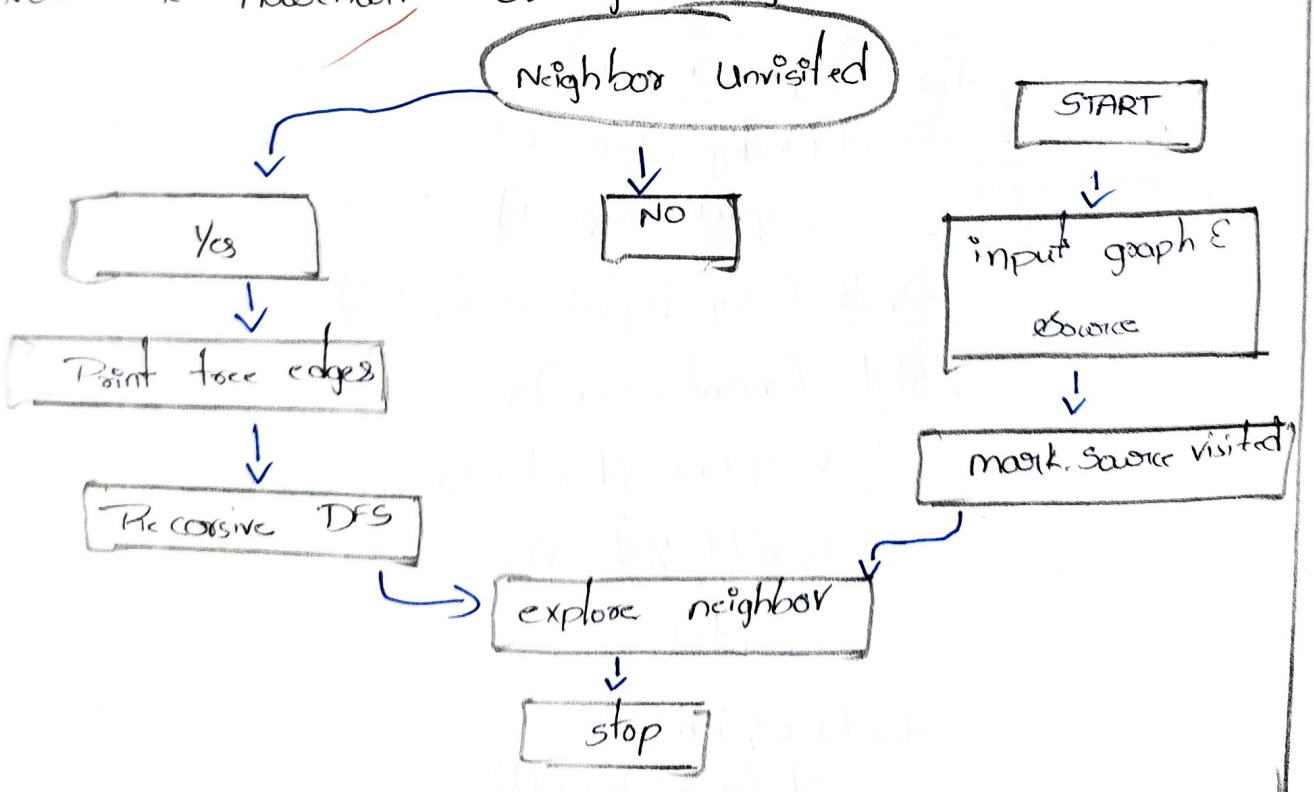
0 1 1 0
0 0 0 1
0 0 0 1
0 0 0 0

Output:

Topological Order: 0 1 2 3

---

(4) Convert the flowchart for generating a DFS tree into code.



Code

```
# include<stdio.h>

int graph[10][10], visited[10], n;

void DFS(int v) {
    int i;
    visited[v] = 1;
    Printf("%d ", v);
    for (i=0; i<n; i++ {
        if (graph[v][i]==1 && visited[i] ==0){

    DFS(i);
    }
}
}

int main() {
    int i, j, start;
    Printf("Enter number of vertices:");
    Scanf("%d", &n);
    for (i=0; i<n; i++)
        for (j=0, j<n; j++)
            Scanf("%d", &graph[i][j]);

Printf("DFS Traversal:");
DFS(start);
return 0;

}
```

Input

5

0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0

Starting vertex : 0

Output

DFS Traversal : 0 1 3 4 2

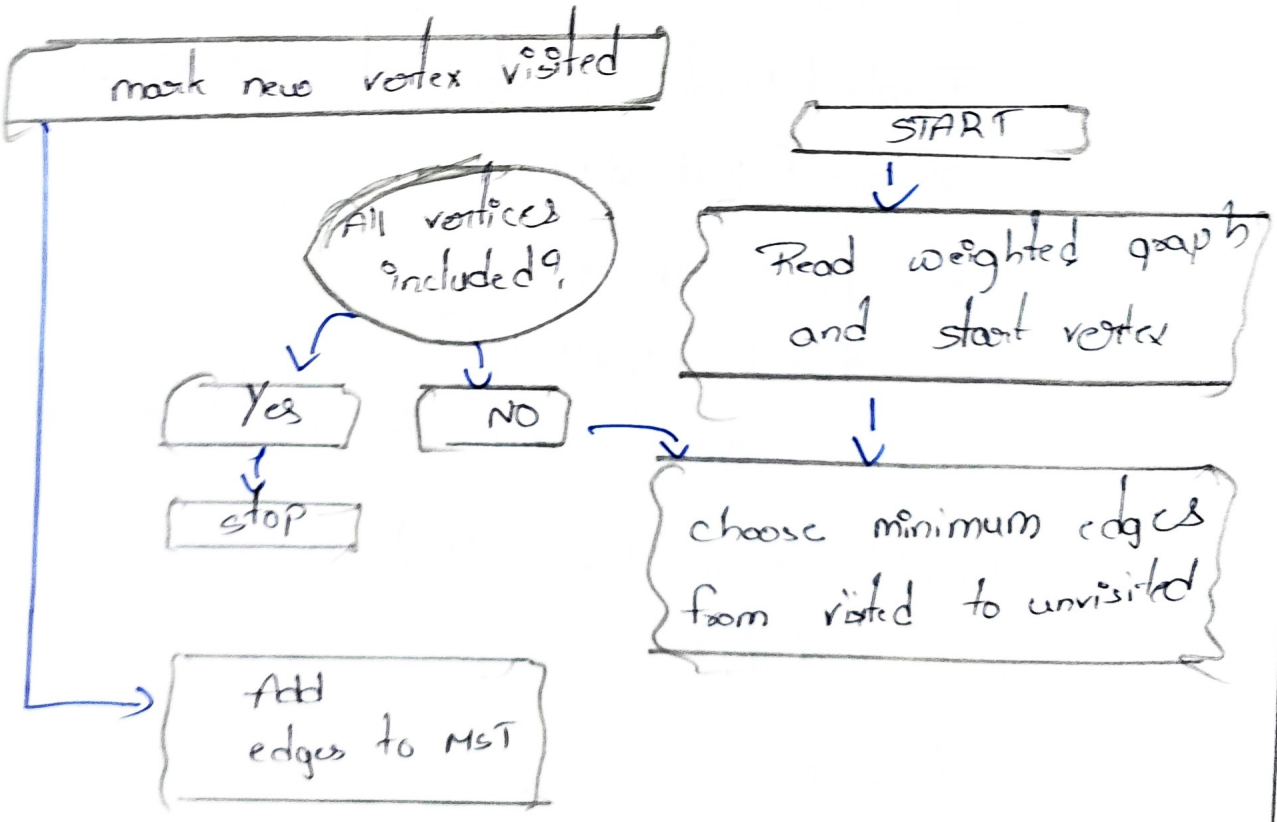
Complexity :

Time : $O(v^2)$

Space : $O(v)$

---

(5) Convert the flowchart for Prim's algorithm into code.



Code:

```
# include <stdio.h>
int main () {
    int cost[10][10], selected[10] = {0};
    int n, i, j, edges=0
    int x, y, min, total=0;
for (i=0 ; i<n ;i++)
    for(j=0 ;j<n ; j++)
        &canf ("%d" &cost[i][j]);
&selected [0]=1;
Printf ("Edges in MST:\n");
while (edge. <n-1) {
    Min =9999;
    for(i=0 ;i<n ;i++){
        if (selected[i] &
            for (j=0 ;i<n ;j++){
                if (!selected[j] && cost[i][j] <min){
    Min = cost[i][j];
        x=i;
        y=j;
    }
}
}
```

```
}
printf("%d - %d = %d\n", x, y, min);
total += min;
selected[y] = 1;
edges ++;
}
printf("Minimum cost = %d\n", total);
return 0;
}
```

Input

4

0 10 6 5

10 0 0 15

6 0 0 4

5 15 4 0

output:

Edges in MST

0-3=5

3-2=4

0-1=10

Minimum cost=19